Real course details

Same layout as the first course. The first hallway, inset, bins, and second hallway are all the same. Other measurements are identical to real_report_2.

Bag command

ros2 bag record -o r<bag name> /diffbot_base_controller/cmd_vel /cmd_vel_raw /diffbot_base_controller/odom /odometry/filtered/local /odometry/filtered/global /global_costmap/costmap /local_costmap/costmap /velodyne_points_nav /imu/data /plan /odom_goal_pose /odom_goal_pose_rviz /tf /dynamic_joint_states /tf_static /local_costmap/published_footprint /global_costmap/published_footprint /rosout /behavior_tree_log /speed_limit /speed_governor_info

Run #1 (Teleoperation, all changes as-is)

bag file: real_bag1

The robot successfully remembered obstacles that fell out of the lidar's view. During the teleop run, the robot actually collided with the second pair of bins during the "forward" and "reverse" sections of the test, and that is reflected in the bag costmap.

The tester raised their left arm to test the decaying logic. It does work, but we want the logic to be

"If a costmap cell is out of the sensor view, decay it normally over 15 seconds"

"If a costmap cell is within view, and we confirm via raytracing that it is no longer occupied, clear it very quickly."

To implement this, we simply increased the decay acceleration significantly in Course2027.yaml (5.0 -> 20.0 in both costmaps).

We also increased the voxel decay rate for unseen obstacles to 30.0 from 15.0 to allow a more generous navigation window.

Run #2 (Teleoperation)

bag file: real_bag2
In this run, we simply drove the robot up to the first pair of bins until the lidar could not see it anymore; then, we waited for the costmap to decay. It *felt* shorter than 30 seconds - I want you to confirm the actual vs. expected decay, quantify the difference, and explain any difference

Run #3 (Teleoperation)

bag name: real_bag3

Same idea as run #2, but with more movement around the obstacle as it was unseen.

Run #4 (Teleoperation)

bag name: real_bag4

Same idea as run #2, but with more deliberate movement to make the obstacle appear/disappear from view. The purpose of all these tele-operated runs is to ensure that the decaying / clearing logic matches what we want.

Run #5 (Autonomous)

bag name: real_bag5

Successfully cleared the course! However, we noticed a few things:

- Would like it to go slightly faster; will increase top_speed later
- The planner failed to plan initially, then waited, planned again, and succeeded. This was not captured in the bag; the bag was started after this all happened. To try and capture this again, the goal pose was re-entered multiple times. We did not observe the same error, but we noticed the planner did oscillate between two goal-approach modes. I assume this is a result of the planner considering any possible heading, so it is expected.
- Will remember to start the bag *before* handing over the goal in the next run.

Run #6 (Autonomous)

Same run as #5 but with the course driven starting from the finish and moving to the start.

Again, a success overall. However, the robot's tail came *very* close to contacting the second pair of bins (the same second pair I have referred to during previous runs). Additionally, during other runs / tests, I noticed that the footprint is *slightly* off from where the robot is in real life in relation to the obstacles; for example, it appears 3cm away IRL but is 4cm away in rviz.

Run #7 (Autonomous, w/ top_speed change)

bag name: real_bag7

Top speed in nav_perception launch changed to default 1.0 and vx_max in Course2027 also changed to 1.0. Rebuilt workspace.

Same as run #5, with the new speed.

Successfully cleared the course!

- We noticed that the robot oscillated on an obstacle approach, maybe due to the high planning frequency causing frequent changes in local path angle…?
- The speed did not increase that much, likely because vx_std was not also raised. Raising that to 0.4.

Run #8 (Autonomous, w/ top_speed change)

bag name: real_bag8

Same as run #6, with the new speed and vx_std raised as well.

Failed to get past the second pair of bins (again, same “second pair” as was labeled in previous forward runs of this course). It swung *very* close to the first bin before stopping.

Run #9 (Autonomous, w/ top_speed change)

bag name: real_bag9

Same as run #8 with std and top speed changes; only a slightly different starting position.

Passed perfectly, and drove exactly as I would. Spot-on, I didn’t observe any significant problems with the physical robot. Worth comparing to run #8 - why did that one fail, but this one succeeded? And analyze the costmap; what was different about that

Questions we want answered

- Why did run #9 succeed and run #8 failed?
- Is the costmap decaying / clearing working as expected? Refer to the svtl layer source code (kilted) and compare that expected behavior to what we saw in the bags.

- We still observe a bit of oscillation on straights. Quantify it and investigate - Is this a software issue, or a system dynamics / latency problem?
- (+everything else that usually goes in the report)
- (+anything else you want)